EFREI Paris – ING2 BDML

Machine Learning I – Année universitaire 2026/2027

Maintenance prédictive d'un banc hydraulique

Prédiction de l'état optimal de la valve à partir des signaux de pression et de débit

---

Rapport technique

[NOM Prénom – à compléter]

[NOM Prénom – à compléter]

Groupe : **[à compléter]**

Enseignants : Ahmed Ghazi, Blaiech, Ahmad Tay, Hassan Shraim

[Date de rendu – à compléter]

Résumé

Ce projet construit un système de maintenance prédictive pour un banc d'essai hydraulique : pour chaque cycle de production de 60 secondes, il prédit si la valve fonctionne de façon optimale (condition = 100 %) ou non. Les seules données utilisées sont la pression PS2 (100 Hz), le débit FS1 (10 Hz) et le fichier des conditions `profile.txt`. Après un contrôle de qualité des données, chaque cycle est résumé par 90 caractéristiques physiques (statistiques globales, dynamique, fenêtres temporelles, énergie spectrale), puis une sélection de variables est apprise à l'intérieur de pipelines `scikit-learn`. Neuf familles de modèles sont comparées en validation croisée stratifiée, optimisées par Grid Search et Random Search, puis le modèle retenu (Régression logistique) est évalué une seule fois sur les 205 derniers cycles : accuracy 0.859, F1 0.834, AUC 1.000. L'AUC parfaite montre que le modèle classe correctement tous les cycles, mais son seuil de décision est décalé par un changement de distribution entre entraînement et test : toutes ses erreurs sont des valves à 90 % prédites optimales, alors que les modèles non linéaires (SVM, forêt aléatoire, gradient boosting) atteignent 100 % sur le test. L'analyse des variables identifie la cause physique d'une valve non optimale : un **retard de la montée en pression** vers 9,4 s dans le cycle (70 ms dès le niveau 90 %). Les bonus réalisés sont présentés (tests unitaires, conteneur Docker, application web).

Table des matières

1 Introduction **2**

1.1 Contexte et objectif . . . 2

1.2 Données utilisées . . . 2

1.3 Organisation du code . . . 2

2 Prétraitement des données **3**

2.1 Chargement et statistiques descriptives . . . 3

2.2 Valeurs manquantes et valeurs aberrantes . . . 3

2.3 Encodage des variables catégorielles . . . 4

2.4 Transformation en classification binaire et équilibre des classes . . . 4

2.5 Visualisation des signaux . . . 5

2.6 Extraction de caractéristiques . . . 6

2.7 Analyse des corrélations . . . 7

2.8 Sélection de variables . . . 9

3 Modélisation **10**

3.1 Pipeline commun . . . 10

3.2 Choix et justification des modèles . . . 10

4 Évaluation des modèles **11**

4.1 Protocole . . . 11

4.2 Comparaison des modèles avant optimisation . . . 11

4.3 Matrices de confusion et courbes ROC . . . 12

4.4 Compromis biais / variance . . . 13

4.5 Robustesse du protocole de validation . . . 14

5 Optimisation des hyperparamètres **14**
5.1 Méthodes . . . 14
5.2 Résultats . . . 15
5.3 Choix du modèle final . . . 16

6 Évaluation finale sur l'échantillon de test **16**
6.1 Analyse des erreurs . . . 17

7 Interprétation : causes d'une valve non optimale **18**

8 Bonus réalisés **20**
8.1 Tests unitaires . . . 20
8.2 Conteneurisation . . . 20
8.3 Application web . . . 21
8.4 Dépôt GitHub . . . 21

9 Limites et perspectives **21**

10 Conclusion **22**

A Reproduire les résultats **22**

1 Introduction

1.1 Contexte et objectif

L'usine souhaite détecter au plus tôt la dégradation de ses machines et comprendre pourquoi la condition de la valve d'un cycle de production est parfois non optimale. On dispose pour cela du jeu de données public *Condition monitoring of hydraulic systems* (UCI, Helwig et al., 2015), obtenu sur un banc d'essai composé d'un circuit principal de travail et d'un circuit secondaire de refroidissement-filtration reliés par le réservoir d'huile. Le banc répète des cycles de charge constants de 60 s pendant lesquels l'état de quatre composants (refroidisseur, valve, pompe, accumulateur) est volontairement dégradé.

Objectif. Construire un modèle de classification binaire qui prédit, pour chaque cycle, si la condition de la valve est optimale (100 %) ou non. Les **2000 premiers cycles** servent à construire le modèle ; les **205 cycles restants** (2001 à 2205) forment l'échantillon de test final.

1.2 Données utilisées

Conformément aux consignes, seuls trois fichiers sont utilisés (tableau 1), chaque ligne correspondant à un cycle.

Table 1 – Fichiers utilisés.

Fichier	Grandeur	Unité	Échantillonnage	Colonnes / cycle
`PS2.txt`	Pression	bar	100 Hz	6000
`FS1.txt`	Débit volumique	l/min	10 Hz	600
`profile.txt`	Conditions des composants	–	1 par cycle	5

Les colonnes de `profile.txt` sont : condition du refroidisseur (3, 20, 100 %), **condition de la valve** (100 % : commutation optimale ; 90 % : léger retard ; 80 % : retard important ; 73 % : proche de la panne), fuite interne de la pompe (0, 1, 2), pression de l'accumulateur (130, 115, 100, 90 bar) et un indicateur de stabilité (0 : conditions stables ; 1 : régime statique peut-être pas encore atteint).

1.3 Organisation du code

Le travail est entièrement reproductible :

- `notebooks/projet_valve.ipynb` : notebook complet (toutes les étapes, figures et tableaux) ;
- `valve_ml/` : module Python (chargement, extraction de caractéristiques, sélection, modèles), réutilisé par les tests et l'application ;
- `tests/`, `Dockerfile`, `app/` : bonus (section 8).

Les figures et tableaux de ce rapport sont exportés automatiquement par le notebook, ce qui garantit que les chiffres cités correspondent exactement à l'exécution.

CAPTURE À INSÉRER

Capture du haut du notebook ouvert dans Jupyter/Colab (titre, plan et cellule d'imports).

`fichier attendu : rapport/captures/notebook_apercu.png`

Figure 1 – Aperçu du notebook dans Jupyter.

2 Prétraitement des données

2.1 Chargement et statistiques descriptives

Les trois fichiers sont lus avec `pandas` (séparateur tabulation, sans en-tête). On obtient deux matrices de signaux de dimensions 2205×6000 (PS2) et 2205×600 (FS1) et un tableau 2205×5 pour les conditions. Le nombre de cycles est vérifié identique dans les trois fichiers.

CAPTURE À INSÉRER

Capture des sorties des cellules « 1.2 Statistiques descriptives » : tableau PS2/FS1 et `profile.describe()`.

`fichier attendu : rapport/captures/stats_descriptives.png`

Figure 2 – Statistiques descriptives des signaux et des conditions (sortie du notebook).

La pression PS2 varie fortement à l'intérieur d'un cycle (montées et descentes de charge), alors que le débit FS1 présente des paliers entrecoupés de transitoires rapides (figure 8). Les conditions des composants sont des niveaux discrets fixés par le plan d'expérience.

2.2 Valeurs manquantes et valeurs aberrantes

Le contrôle qualité porte sur les valeurs manquantes, les valeurs infinies, les valeurs physiquement impossibles (pression ou débit négatifs), les cycles constants (capteur figé) et les lignes dupliquées.

CAPTURE À INSÉRER

Capture du tableau « NaN / inf / valeurs < 0 / cycles constants / lignes dupliquées » et du tableau de détection des valeurs aberrantes (section 1.3).

`fichier attendu : rapport/captures/qualite_donnees.png`

Figure 3 – Contrôle de qualité des données (sortie du notebook).

Aucune valeur manquante ni infinie n'est présente : aucune imputation n'est nécessaire. Pour les valeurs aberrantes, on calcule pour chaque cycle la moyenne, l'écart-type et le maximum de chaque signal, puis on applique la règle de l'IQR ($x < Q_1 - 1{,}5\,\mathrm{IQR}$ ou $x > Q_3 + 1{,}5\,\mathrm{IQR}$) et un z-score robuste (médiane et MAD, seuil 3,5). 331 cycles sont signalés par au moins un critère IQR (figure 4).

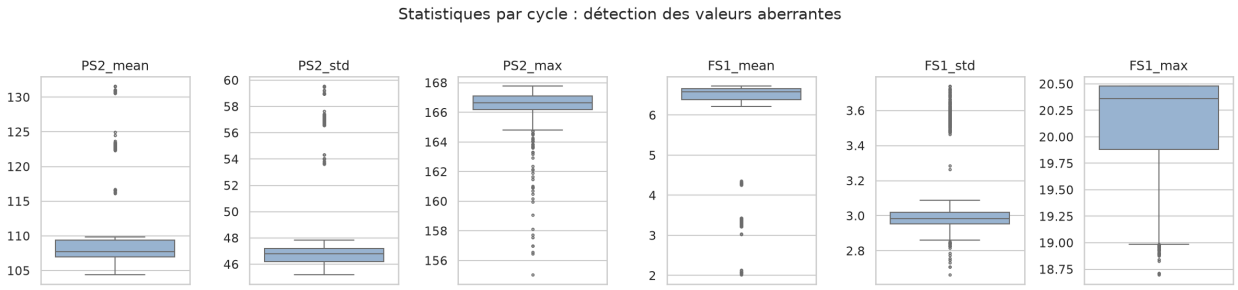


Figure 4 – Boxplots des statistiques par cycle utilisés pour détecter les valeurs aberrantes.

Décision : les cycles atypiques sont conservés. Ce ne sont pas des erreurs de mesure mais des états physiques réels du banc (refroidisseur proche de la panne, accumulateur dégradé, régime transitoire). Les supprimer retirerait précisément les situations qu'un système de maintenance doit reconnaître, et de tels cycles existent dans le jeu de test. Leur influence est maîtrisée par la standardisation et par l'utilisation de modèles robustes (arbres et ensembles).

2.3 Encodage des variables catégorielles

Les capteurs ne fournissent que des variables continues. Les colonnes de `profile.txt` sont des niveaux ordinaux déjà numériques ; elles ne sont pas utilisées comme variables explicatives car ce sont des étiquettes d'état mesurées hors ligne, inconnues en production (les utiliser constituerait une fuite d'information). Aucun encodage one-hot n'est donc nécessaire ; la seule transformation est la binarisation de la cible.

2.4 Transformation en classification binaire et équilibre des classes

La cible est définie par

$$y = \begin{cases} 1 & \text{si la condition de la valve vaut } 100\,\% \text{ (optimale)}, \\ 0 & \text{sinon (90, 80 ou } 73\,\%). \end{cases}$$

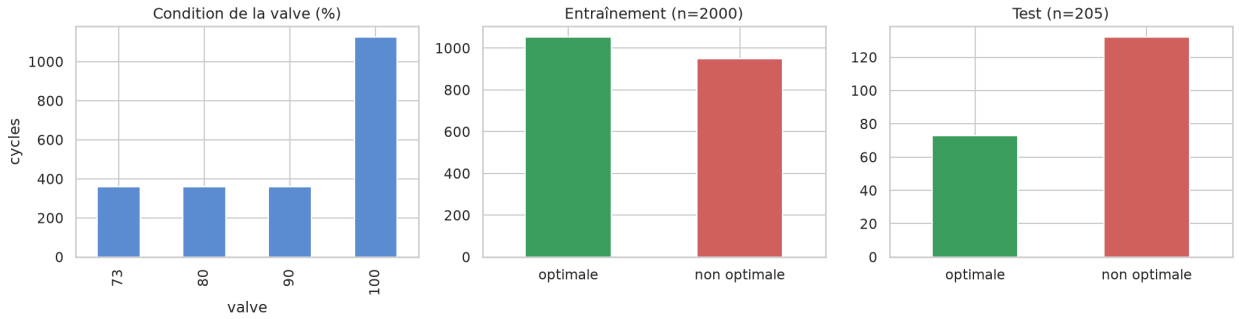


Figure 5 – Distribution des niveaux de la valve et des classes binaires (entraînement / test).

Le jeu d'entraînement est quasiment équilibré (52.6 % de cycles optimaux) : aucun rééchantillonnage n'est nécessaire, mais la validation croisée est stratifiée et l'on suit le F1-score et l'AUC en plus de l'accuracy. Le jeu de test est déséquilibré dans l'autre sens (35.6 % de cycles optimaux, soit 73 sur 205).

Le découpage imposé est **temporel**. Or les conditions des autres composants évoluent au cours de l'expérience (figure 6) : les 205 cycles de test ont tous un refroidisseur à 100 % et un accumulateur majoritairement à 90 bar. Le jeu de test n'est donc pas distribué comme le jeu d'entraînement (*dataset shift*), ce qui rend l'évaluation finale plus exigeante qu'une validation aléatoire.

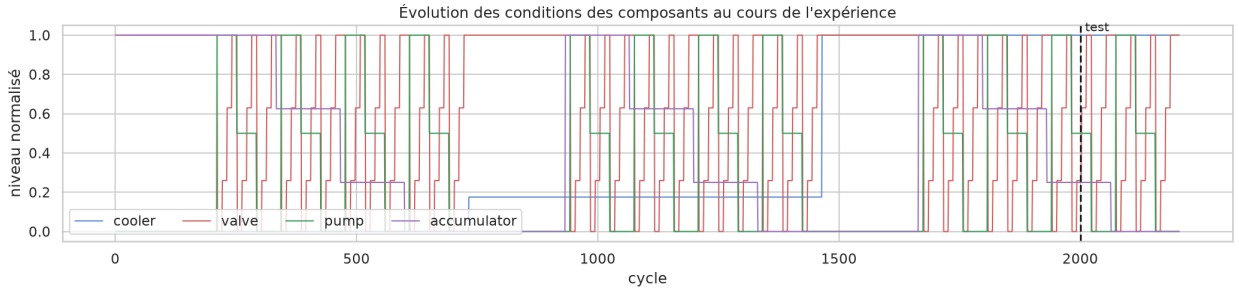


Figure 6 – Évolution des conditions des composants au fil des cycles ; la ligne pointillée sépare entraînement et test.

2.5 Visualisation des signaux

La figure 7 superpose le profil moyen d'un cycle pour chaque niveau de la valve. Le cycle comporte une phase sans pression (0 à 9 s environ), une forte montée en pression vers 9,4 s accompagnée d'un pic de débit, puis des paliers de charge qui changent toutes les 10 s. Deux différences apparaissent nettement :

— la **montée en pression et le pic de débit arrivent d'autant plus tard que la valve est dégradée** : c'est la signature directe d'un retard de commutation ;
— sur les paliers suivants, la pression est légèrement plus haute et le débit plus bas quand la valve est dégradée (pertes de charge plus importantes).

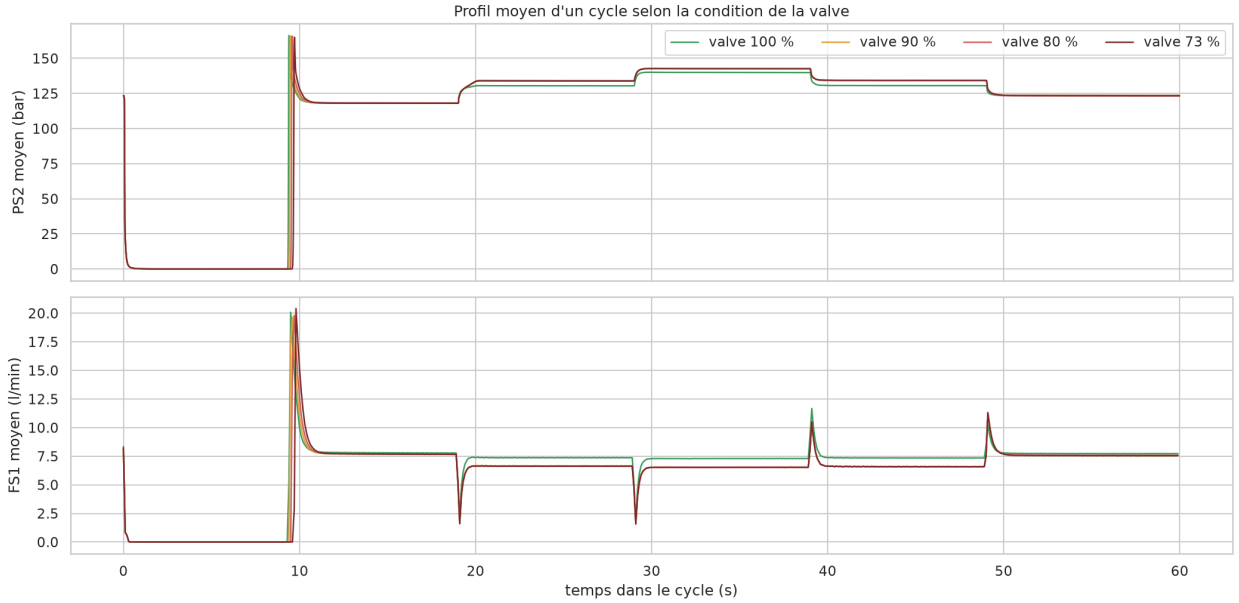


Figure 7 – Profil moyen de PS2 et FS1 sur un cycle selon la condition de la valve.

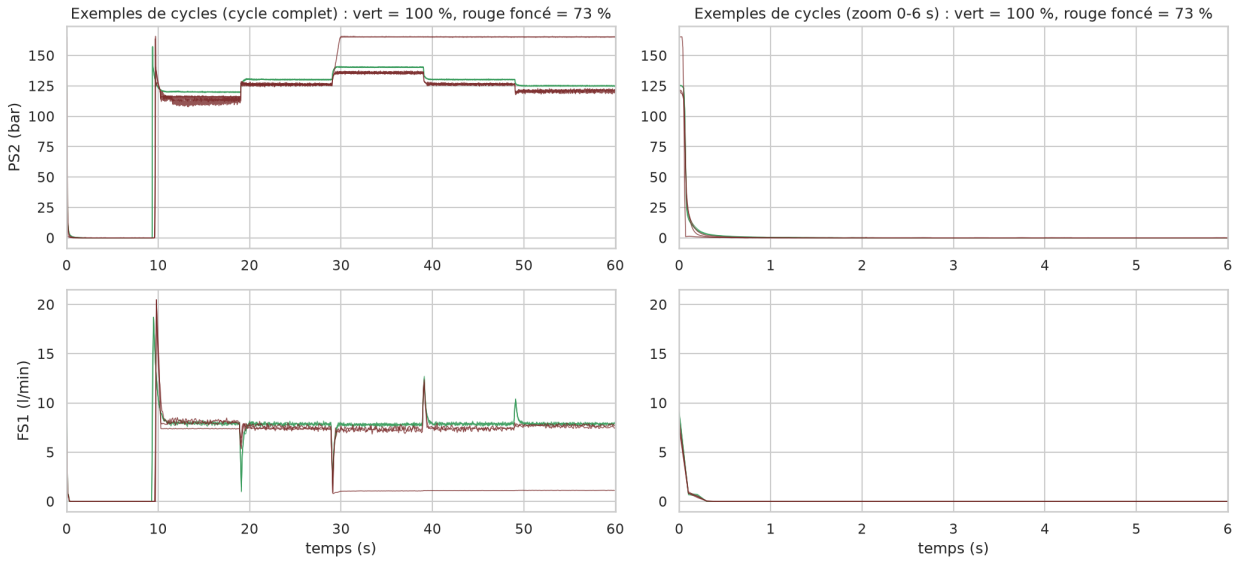


Figure 8 – Exemples de cycles bruts (vert : valve à 100 % ; rouge foncé : 73 %), cycle complet et zoom sur les 6 premières secondes.

2.6 Extraction de caractéristiques

Les 6600 points bruts d'un cycle sont très redondants et inadaptés à des modèles comme k-NN, Naive Bayes ou un arbre unique. Chaque signal est donc résumé par des descripteurs interprétables (tableau 2), soit 90 variables au total.

Table 2 – Caractéristiques extraites pour chaque signal (PS2 et FS1).

Famille	Descripteurs	Nombre
Statistiques globales	moyenne, écart-type, min, max, médiane, quantiles 5/25/75/95 %, IQR, amplitude, RMS, asymétrie, aplatissement, instants du max et du min	16
Dynamique	moyenne de $\lvert dx/dt \rvert$, écart-type, max et min de la dérivée	4
Fenêtres temporelles	moyenne et écart-type sur 10 fenêtres de 6 s	20
Spectre	part d'énergie dans 4 bandes logarithmiques, énergie totale (log)	5
Total par signal		45

CAPTURE À INSÉRER

Capture de la cellule du notebook qui définit `extract_signal_features` et de la sortie « Matrice de caractéristiques : (2205, 90) ».

`fichier attendu : rapport/captures/code_features.png`

Figure 9 – Code de l'extraction de caractéristiques (module `valve_ml/features.py`).

2.7 Analyse des corrélations

Toutes les analyses à partir de cette section sont faites sur les 2000 cycles d'entraînement uniquement, pour ne pas laisser d'information du test influencer les choix.

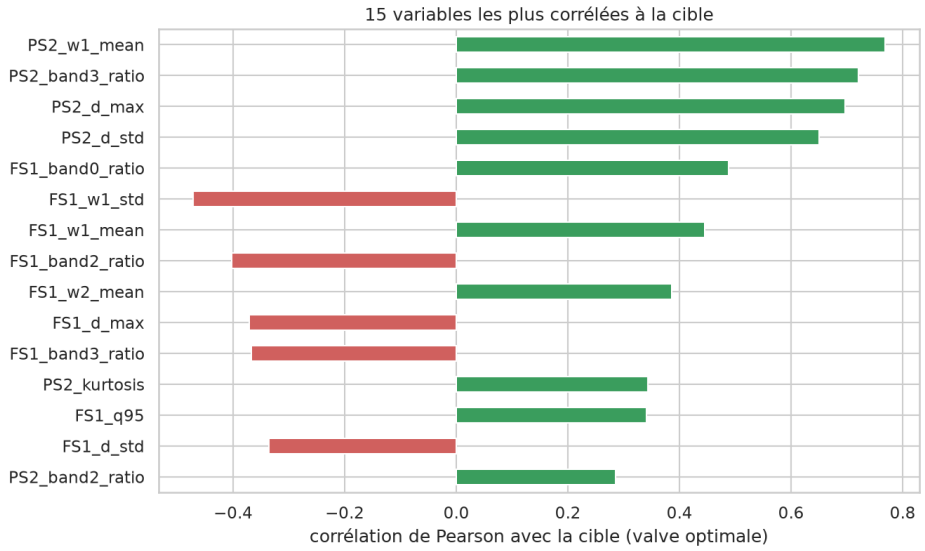


Figure 10 – Les 15 variables les plus corrélées (Pearson) à la cible.

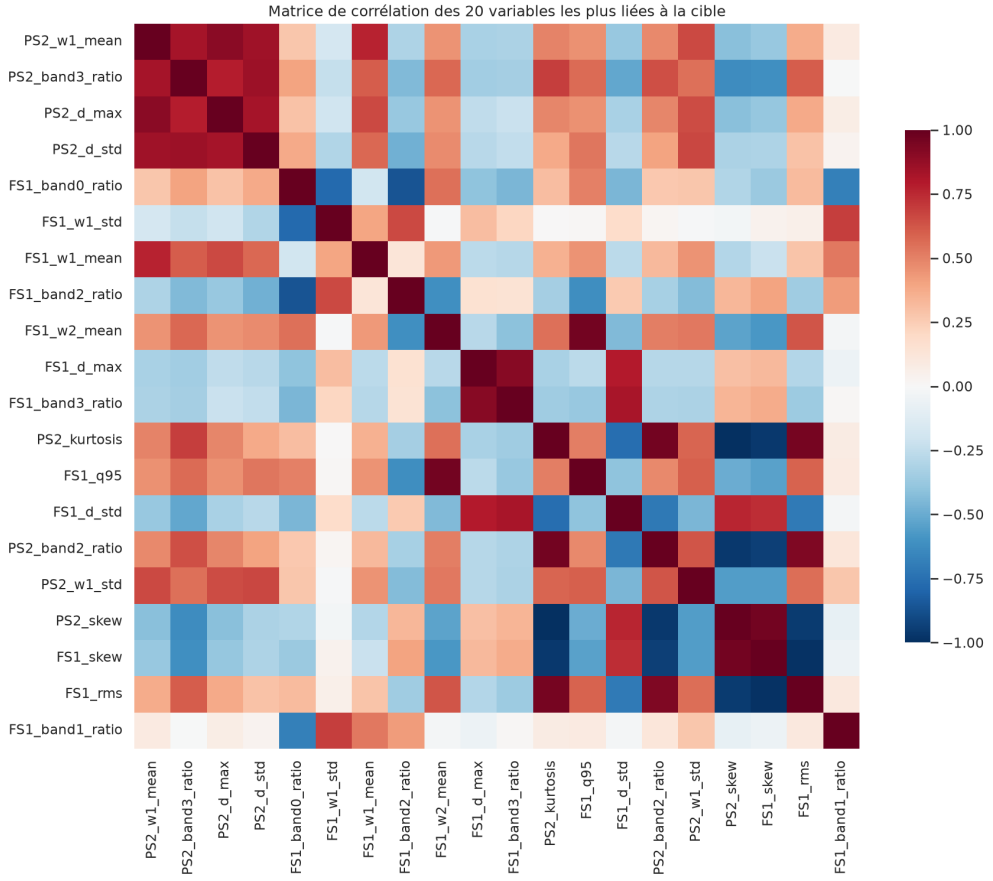


Figure 11 – Matrice de corrélation des 20 variables les plus liées à la cible.

La matrice de corrélation montre de forts blocs de redondance : 11.3 % des paires de variables ont une corrélation absolue supérieure à 0,95 (par exemple moyenne, médiane et RMS d'un même signal). Cette colinéarité gêne la régression logistique (coefficients instables) et viole l'hypothèse d'indépendance de Naive Bayes, d'où le filtre de corrélation de la section suivante. Les histogrammes et boxplots de la figure 12 montrent que plusieurs variables séparent nettement les deux classes. Les plus corrélées à la cible sont issues de la pression PS2 : moyenne sur la fenêtre 6–12 s (`PS2_w1_mean`, $r = 0{,}77$), part d'énergie haute fréquence (`PS2_band3_ratio`, $r = 0{,}72$) et raideur maximale de la montée (`PS2_d_max`, $r = 0{,}70$). Toutes décrivent la montée en pression autour de 9,4 s. Les variables de débit FS1 sont moins corrélées ($|r| < 0{,}5$) mais apportent une information complémentaire.

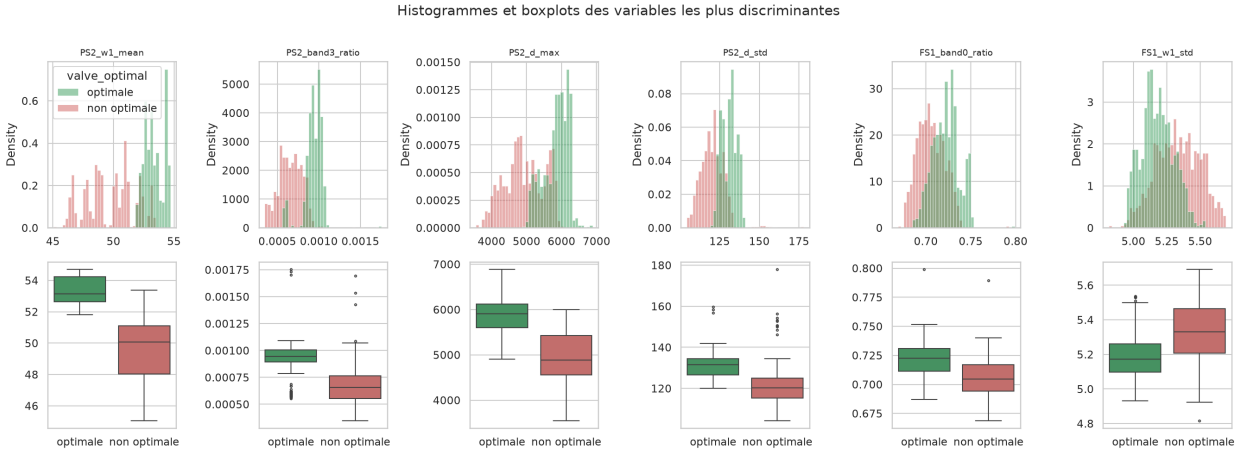


Figure 12 – Histogrammes (haut) et boxplots (bas) par classe des six variables les plus discriminantes.

Lien avec les autres composants. La figure 13 et le V de Cramér (tableau 3) mesurent l'association entre la condition de la valve et celles des autres composants.

Table 3 – Association entre la condition de la valve et les autres variables de `profile.txt` (V de Cramér, 2205 cycles).

	Refroidisseur	Pompe	Accumulateur	Indicateur de stabilité
V de Cramér	0,004	0,307	0,171	0,708

Le refroidisseur est indépendant de la valve. Les associations avec la pompe, l'accumulateur et surtout l'indicateur de stabilité viennent du **plan d'expérience** et non d'un lien physique. En effet, chaque niveau dégradé de la valve (73, 80, 90 %) est testé exactement 120 fois pour chaque état de la pompe et 90 fois pour chaque état de l'accumulateur ; de plus, les 756 cycles « non stables » (phases de transition après un changement de configuration) ont tous été enregistrés avec une valve à 100 %. Ces variables ne sont donc pas des causes de la dégradation de la valve, et elles ne seraient de toute façon pas disponibles en production. La dégradation doit être détectée dans la **signature des signaux** de pression et de débit.

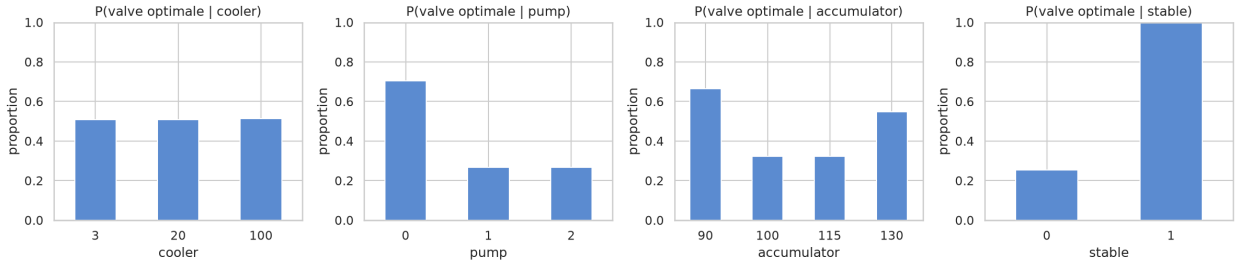


Figure 13 – Proportion de cycles avec valve optimale selon l'état des autres composants.

2.8 Sélection de variables

Quatre familles de méthodes sont combinées :

1. **filtre de variance** : suppression des variables constantes ;
2. **filtre de corrélation** : pour chaque paire avec $|r| > 0{,}95$ une seule variable est gardée (90 $\rightarrow$ 47 variables) ;
3. **méthodes univariées** : test ANOVA F et information mutuelle ;
4. **méthodes embarquées** : régression logistique pénalisée L1 (13 coefficients non nuls pour $C = 0{,}1$) et importance des variables d'une forêt aléatoire.

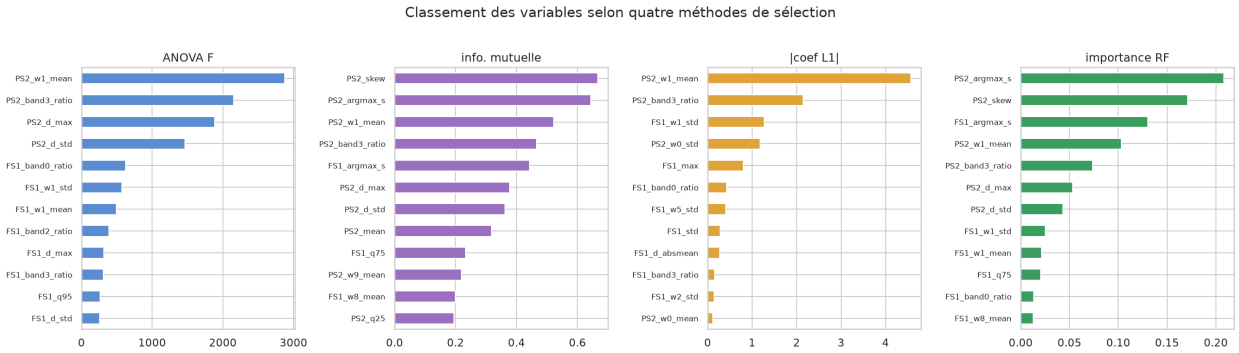


Figure 14 – Classement des variables selon l'ANOVA F, l'information mutuelle, la pénalisation L1 et la forêt aléatoire.

Les méthodes s'accordent sur les deux variables les plus utiles, `PS2_w1_mean` et `PS2_band3_ratio` (meilleurs rangs moyens), ce qui renforce la confiance dans la sélection. Elles diffèrent ensuite : l'information mutuelle et la forêt aléatoire valorisent des variables à relation non monotone avec la cible (asymétrie, instant du maximum), que le test ANOVA F, qui ne mesure qu'un écart de moyennes, classe plus bas. La pénalisation L1 ne garde que 13 variables, ce qui confirme que l'information est concentrée dans peu de descripteurs. Pour la modélisation, le filtre de corrélation et la sélection `SelectKBest` (ANOVA F) sont placés **à l'intérieur du pipeline** : ils sont réappris dans chaque pli de validation croisée, ce qui évite toute fuite d'information. Le nombre de variables k est choisi en traçant l'AUC en validation croisée en fonction de k (figure 15) : on retient par défaut le plus petit k dont l'AUC est à moins de 0,005 du maximum, soit $k = 10$; au-delà, l'AUC est saturée (proche de 1). Ensuite k est de nouveau optimisé avec les hyperparamètres (section 5).

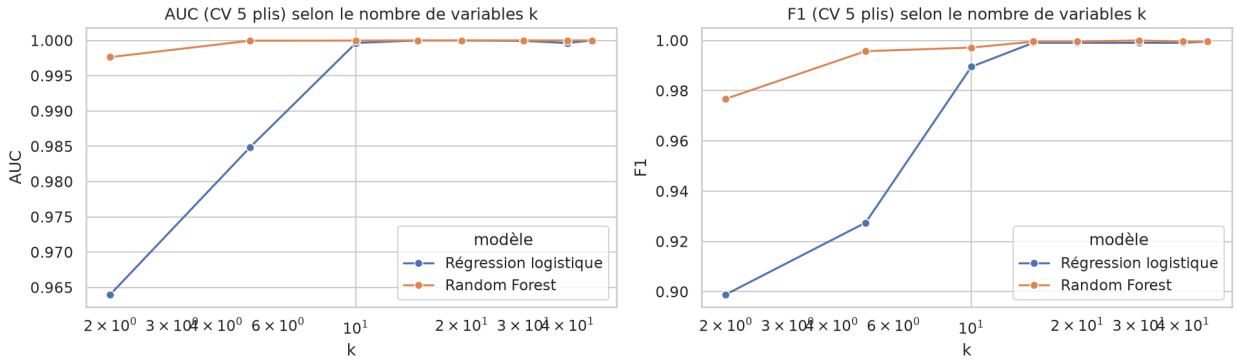


Figure 15 – AUC et F1 en validation croisée selon le nombre de variables retenues.

3 Modélisation

3.1 Pipeline commun

Tous les modèles partagent le même enchaînement, ce qui rend la comparaison équitable :

`CorrelationFilter(0,95)` → `StandardScaler` → `SelectKBest(ANOVA F, k)` → classifieur.

La standardisation est indispensable pour k-NN, le SVM et la régression logistique ; elle est neutre pour les arbres.

3.2 Choix et justification des modèles

Le tableau 4 résume les propriétés théoriques des modèles comparés et leur adéquation aux données : variables continues, en partie corrélées, avec un bruit de mesure et des cycles atypiques.

Table 4 – Modèles comparés et justification.

Modèle	Nature	Adaptation aux données
Régression logistique	linéaire, paramétrique	Référence simple et interprétable. Suffisante si les classes sont séparables linéairement dans l'espace des caractéristiques ; sensible à la colinéarité (traitée par le filtre de corrélation et la régularisation L1/L2).
k-NN	non linéaire, à base d'instances	Frontière locale très flexible, sans hypothèse de distribution ; sensible aux échelles (standardisation), au bruit et à la dimension (d'où la sélection de variables).
Arbre de décision	non linéaire, non paramétrique	Insensible aux échelles et aux valeurs extrêmes, règles lisibles ; variance élevée (sur-apprentissage) si la profondeur n'est pas limitée.
Naive Bayes gaussien	probabiliste, génératif	Très rapide ; suppose des variables indépendantes et gaussiennes, hypothèses violées ici (variables corrélées, distributions multimodales) : sert de référence à biais élevé.
SVM (noyau RBF)	non linéaire, marge maximale	Bien adapté aux variables continues standardisées de dimension moyenne ; la marge le rend robuste au bruit ; demande le réglage de C et γ.
Random Forest	ensemble (bagging)	Moyenne d'arbres décorrélés : réduit la variance, robuste au bruit, aux valeurs aberrantes et aux variables corrélées.
Gradient Boosting, AdaBoost	ensemble (boosting)	Corrige séquentiellement les erreurs : réduit le biais ; doit être régularisé (taux d'apprentissage, profondeur, sous-échantillonnage).
Stacking	ensemble hétérogène	Combine des modèles de biais différents (linéaire, noyau, arbres) par un méta-modèle logistique.

4 Évaluation des modèles

4.1 Protocole

Les modèles sont évalués par **validation croisée stratifiée à 5 plis** (mélangés, graine 42) sur les 2000 cycles d'entraînement. Pour chaque modèle on mesure l'accuracy, la précision, le rappel, le F1-score (classe positive : valve optimale) et l'aire sous la courbe ROC (AUC). Les scores sur les plis d'entraînement sont aussi relevés : l'écart entraînement–validation mesure la variance, le niveau du score de validation mesure le biais.

4.2 Comparaison des modèles avant optimisation

Table 5 – Validation croisée (5 plis) des modèles avec leurs hyperparamètres par défaut. « F1 train » est le F1 sur les plis d'entraînement, « Écart F1 » l'écart entraînement–validation.

Modèle	Accuracy	Précision	Rappel	F1	AUC	F1 train	Écart F1
Random Forest	0.997	0.997	0.997	0.997	1.000	1.000	0.003
Stacking (LR+SVM+RF)	0.995	0.993	0.996	0.995	1.000	0.997	0.002
SVM (RBF)	0.995	0.994	0.995	0.995	1.000	0.996	0.001
Gradient Boosting	0.996	0.996	0.997	0.997	1.000	1.000	0.003
Régression logistique	0.989	0.990	0.989	0.990	1.000	0.991	0.001
k-NN	0.994	0.995	0.992	0.994	0.999	0.997	0.003
AdaBoost	0.991	0.991	0.991	0.991	0.999	1.000	0.008
Arbre de décision	0.987	0.988	0.987	0.987	0.986	1.000	0.013
Naive Bayes	0.901	0.899	0.916	0.907	0.976	0.907	0.000

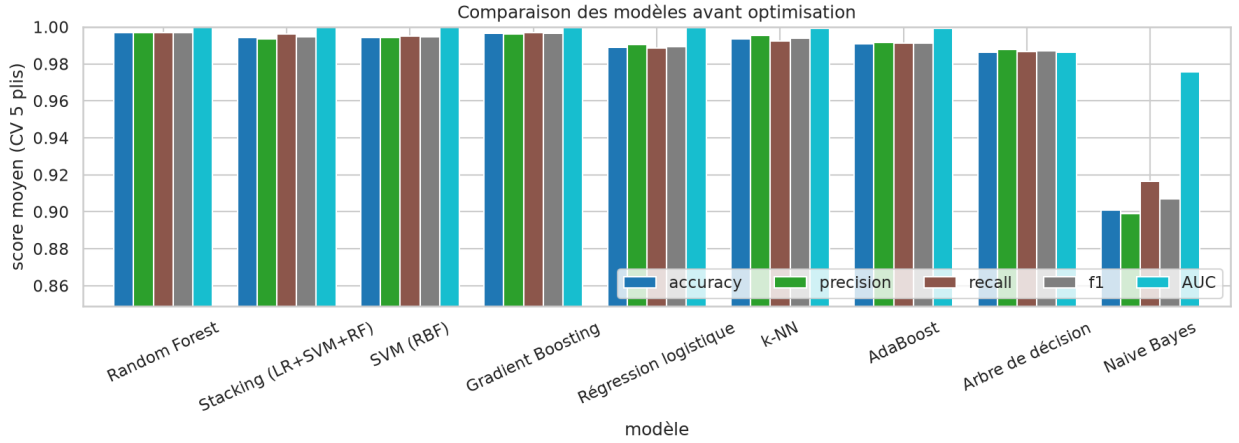


Figure 16 – Comparaison des métriques en validation croisée avant optimisation.

Le problème est très bien séparé dans l'espace des caractéristiques : tous les modèles sauf Naive Bayes dépassent 0,98 de F1 et 0,98 d'AUC. Les ensembles (forêt aléatoire, gradient boosting) et le SVM sont les meilleurs (F1 $\approx$ 0,995–0,997). La régression logistique, modèle linéaire, atteint déjà 0,99 de F1 : la frontière entre les classes est presque linéaire. Naive Bayes reste nettement en dessous (F1 $\approx$ 0,91) car ses hypothèses d'indépendance et de normalité sont violées. L'arbre de décision non élagué a l'écart entraînement–validation le plus grand (0,013), signe de variance.

4.3 Matrices de confusion et courbes ROC

Les prédictions hors-pli (`cross_val_predict`) permettent de construire, pour chaque modèle, une matrice de confusion et une courbe ROC sur l'ensemble des 2000 cycles d'entraînement, chaque cycle étant prédit par un modèle qui ne l'a pas vu.

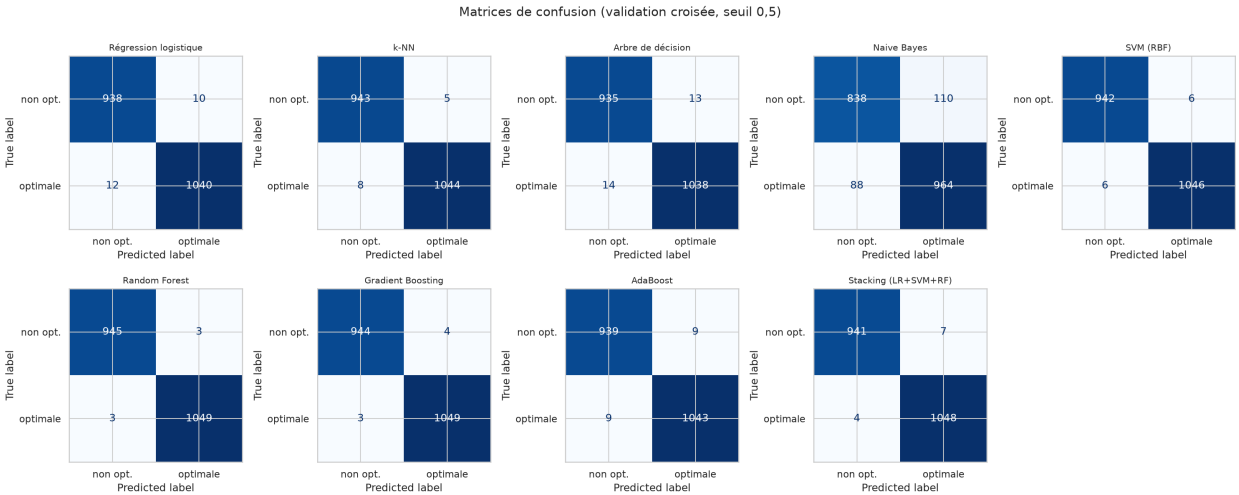


Figure 17 – Matrices de confusion en validation croisée (seuil 0,5).

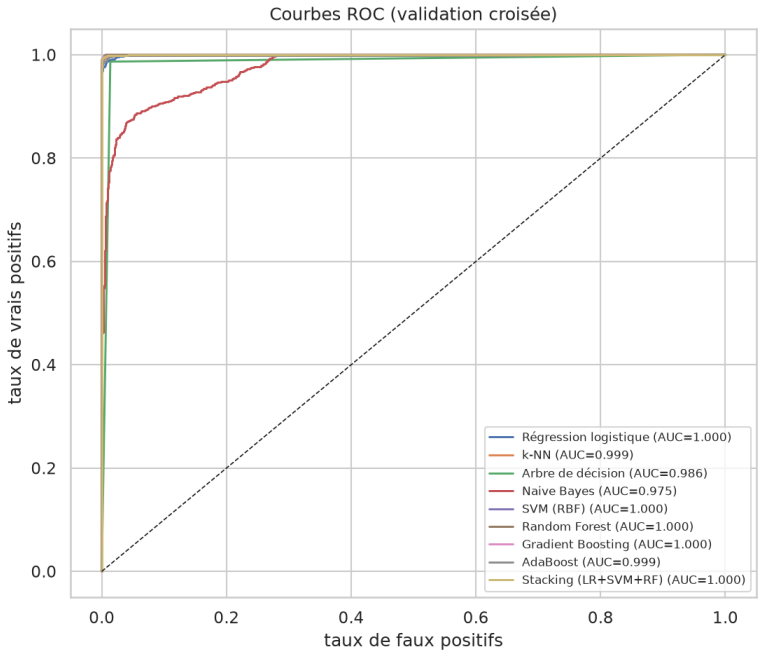


Figure 18 – Courbes ROC en validation croisée.

4.4 Compromis biais / variance

Les courbes d'apprentissage (figure 19) se lisent ainsi : un modèle à forte **variance** (arbre non élagué, k-NN avec peu de voisins) a un score d'entraînement proche de 1 et un écart important avec la validation ; un modèle à fort **biais** (Naive Bayes) a des courbes d'entraînement et de validation basses et proches. Le bon compromis est obtenu par les modèles dont la courbe de validation est haute et proche de la courbe d'entraînement.

Ici, l'arbre de décision et la forêt aléatoire ont un score d'entraînement de 1 quelle que soit la taille du jeu : ils mémorisent les données. Pour l'arbre, l'écart reste visible même avec 1600 cycles (variance élevée) ; la forêt réduit cet écart grâce au bagging. Naive Bayes présente deux courbes basses et confondues autour de 0,90 : c'est un biais élevé, que plus de données ne corrigera pas. La régression logistique, le SVM et k-NN ont des courbes qui convergent vers 0,99–1 avec un faible écart : c'est le meilleur compromis biais/variance.

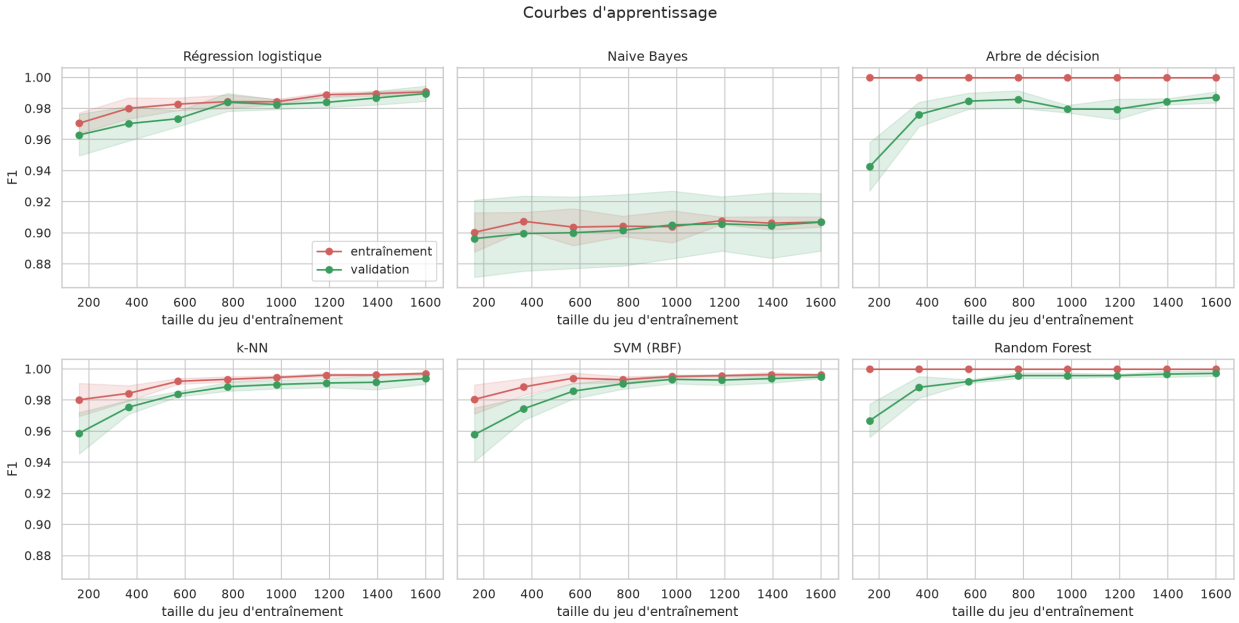


Figure 19 – Courbes d'apprentissage (F1) : entraînement en rouge, validation en vert.

4.5 Robustesse du protocole de validation

Les cycles successifs partagent les mêmes conditions de fonctionnement, donc une validation à plis mélangés place dans le pli de validation des cycles très semblables à ceux de l'entraînement. Le tableau 6 compare cette validation à une validation à **plis contigus** (sans mélange), plus proche de la situation réelle où l'on prédit des cycles futurs.

Table 6 – Validation à plis mélangés vs plis contigus.

Modèle	F1 plis mélangés	F1 plis contigus	AUC plis mélangés	AUC plis contigus
Régression logistique	0.990	0.979	1.000	0.999
SVM (RBF)	0.995	0.978	1.000	0.999
Random Forest	0.997	0.935	1.000	0.994
Gradient Boosting	0.997	0.925	1.000	0.986

Le résultat est net : avec des plis contigus, le F1 des ensembles d'arbres chute (0,997 → 0,935 pour la forêt aléatoire, 0,997 → 0,925 pour le gradient boosting) alors que la régression logistique et le SVM restent vers 0,98. La validation mélangée est donc optimiste, et les performances sur une période non vue dépendent de la capacité du modèle à extrapoler à des conditions de fonctionnement nouvelles.

5 Optimisation des hyperparamètres

5.1 Méthodes

— **Grid Search** (recherche exhaustive) pour les modèles à espace de recherche réduit : régression logistique (C, pénalité L1/L2), k-NN (nombre de voisins, pondération, distance), arbre de décision (profondeur, taille minimale des feuilles, critère), Naive Bayes (lissage de variance) et SVM (C, γ) ;
— **Random Search** (40 tirages) pour Random Forest et Gradient Boosting, dont l'espace (nombre d'arbres, profondeur, taux d'apprentissage, sous-échantillonnage…) est trop grand

pour une recherche exhaustive.

Le nombre de variables k est optimisé conjointement. Le critère est l'AUC en validation croisée stratifiée à 5 plis ; le pipeline complet (filtre, standardisation, sélection, modèle) est réappris dans chaque pli. Enfin, un **stacking** des trois meilleurs modèles optimisés est construit.

CAPTURE À INSÉRER

Capture de la sortie de la cellule d'optimisation (section 4) :
une ligne par modèle avec AUC, durée et meilleurs paramètres.

`fichier attendu : rapport/captures/gridsearch_sortie.png`

Figure 20 – Exécution des recherches d'hyperparamètres (sortie du notebook).

5.2 Résultats

Table 7 – Gain apporté par l'optimisation (AUC en validation croisée).

Modèle	Méthode	Comb.	AUC avant	AUC après	Temps (s)
Régression logistique	Grid	60	1.000	1.000	1.857
k-NN	Grid	168	0.999	1.000	5.381
Arbre de décision	Grid	240	0.986	0.999	8.321
Naive Bayes	Grid	54	0.976	0.976	1.315
SVM (RBF)	Grid	96	1.000	1.000	20.042
Random Forest	Random	40	1.000	1.000	57.095
Gradient Boosting	Random	40	1.000	1.000	123.504

Table 8 – Meilleurs hyperparamètres trouvés.

Modèle	Meilleurs hyperparamètres
Régression logistique	`C=0.1, penalty=l1, solver=liblinear, k=20`
k-NN	`n_neighbors=7, p=2, weights=distance, k=5`
Arbre de décision	`criterion=gini, max_depth=3, min_samples_leaf=5, k=20`
Naive Bayes	`var_smoothing=0.001, k=10`
SVM (RBF)	`C=10, gamma=0.001, k=40`
Random Forest	`max_depth=10, max_features=log2, min_samples_leaf=3, n_estimators=314, k=20`
Gradient Boosting	`learning_rate=0.07357, max_depth=2, n_estimators=351, subsample=0.8428, k=40`

Table 9 – Validation croisée des modèles optimisés.

Modèle	Accuracy	Précision	Rappel	F1	AUC	F1 train	Écart F1
Régression logistique	0.994	0.988	1.000	0.994	1.000	0.994	0.001
Stacking optimisé	1.000	0.999	1.000	1.000	1.000	1.000	0.000
Random Forest	1.000	1.000	0.999	1.000	1.000	1.000	0.000
SVM (RBF)	0.996	0.993	1.000	0.997	1.000	0.997	0.000
Gradient Boosting	0.997	0.998	0.996	0.997	1.000	1.000	0.003
k-NN	0.995	1.000	0.990	0.995	1.000	1.000	0.005
Arbre de décision	0.997	0.997	0.997	0.997	0.999	0.999	0.002
Naive Bayes	0.902	0.899	0.917	0.907	0.976	0.907	-0.000

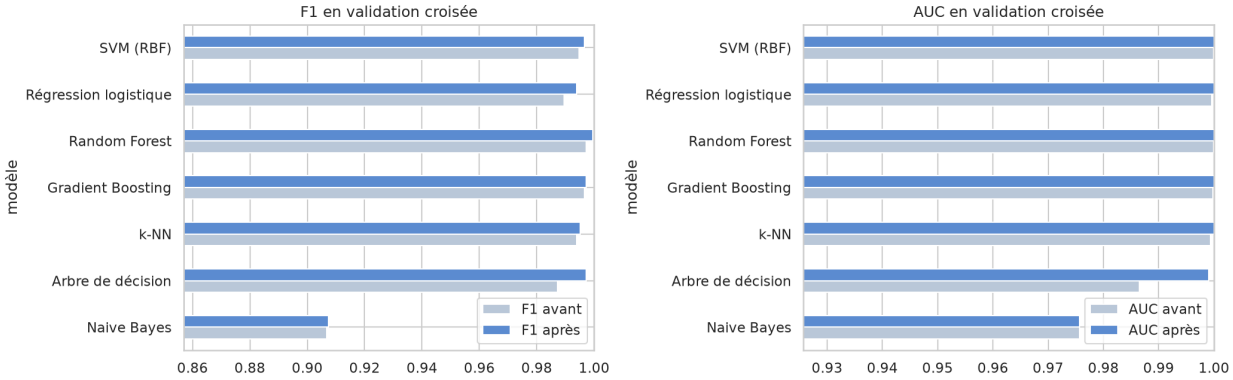


Figure 21 – F1 et AUC en validation croisée avant et après optimisation.

5.3 Choix du modèle final

On retient le modèle ayant la meilleure AUC en validation croisée ; en cas de quasi-égalité (écart inférieur à 0,002) on préfère le modèle le plus simple puis celui dont l'écart entraînement–validation est le plus faible, c'est-à-dire le meilleur compromis biais/variance. Le modèle retenu est **Régression logistique** (AUC en validation croisée 1.000, F1 0.994, accuracy 0.994), avec une pénalité L1 ($C = 0{,}1$) et $k = 20$ variables.

Après optimisation, presque tous les modèles atteignent une AUC de 1,000 en validation croisée : le critère est **saturé** et ne permet plus de les départager. La règle de départage choisit alors le modèle le plus simple, qui a aussi le plus petit écart entraînement–validation et les meilleurs scores en validation à plis contigus (tableau 6). Ce choix est cohérent avec toutes les informations disponibles avant le test.

6 Évaluation finale sur l'échantillon de test

Le modèle final est entraîné sur les 2000 premiers cycles puis évalué une seule fois sur les cycles 2001 à 2205. Les autres modèles optimisés sont évalués à titre de comparaison seulement : le choix a été fait avant, sur la validation croisée.

Table 10 – Performances du modèle final (Régression logistique) sur le test.

Accuracy	Précision	Rappel	F1	AUC
0.859	0.716	1.000	0.834	1.000

	prédit non opt.	prédit optimale
réel non optimale	103	29
réel optimale	0	73

Table 11 – Performances de tous les modèles optimisés sur le test (comparaison).

Modèle	Accuracy	Précision	Rappel	F1	AUC
Régression logistique	0.859	0.716	1.000	0.834	1.000
Arbre de décision	1.000	1.000	1.000	1.000	1.000
Random Forest	1.000	1.000	1.000	1.000	1.000
SVM (RBF)	1.000	1.000	1.000	1.000	1.000
Gradient Boosting	1.000	1.000	1.000	1.000	1.000
Stacking optimisé	0.902	0.785	1.000	0.880	1.000
k-NN	0.946	0.869	1.000	0.930	0.996
Naive Bayes	0.659	0.510	1.000	0.676	0.909

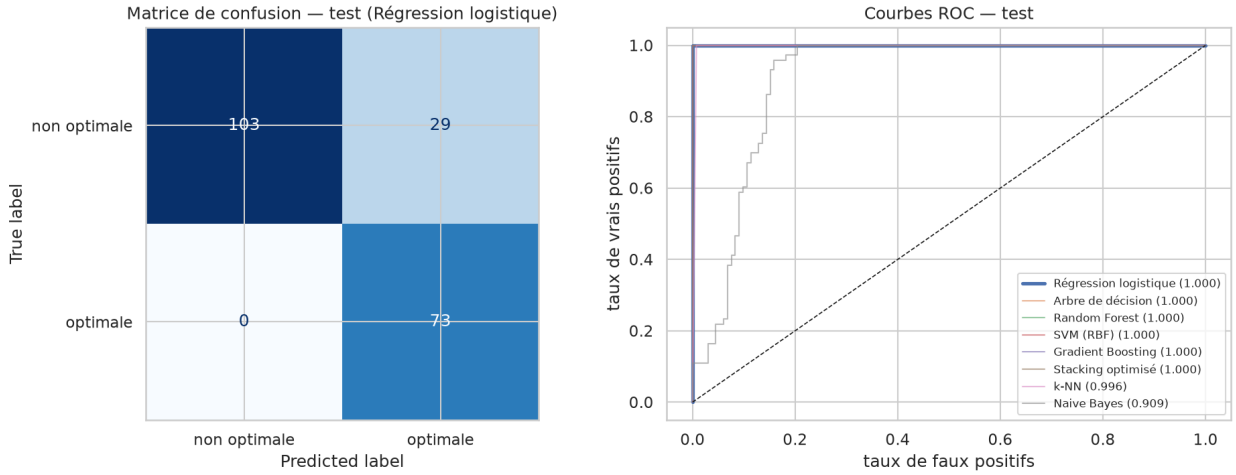


Figure 22 – Matrice de confusion du modèle final et courbes ROC de tous les modèles sur le test.

Le modèle final ne fait **aucun faux négatif** (rappel 1) mais 29 faux positifs : des cycles à valve non optimale prédits optimaux. Son AUC sur le test vaut pourtant 1.000 : les probabilités *classent* parfaitement les cycles, mais le seuil de 0,5 appris sur l'entraînement n'est plus adapté au test. À l'inverse, le SVM, la forêt aléatoire, le gradient boosting et l'arbre de décision optimisés obtiennent 100 % sur le test (tableau 11).

6.1 Analyse des erreurs

Le tableau 12 détaille les erreurs selon le niveau réel de la valve et la figure 23 montre la probabilité prédite pour chaque cycle de test.

Table 12 – Erreurs du modèle final sur le test selon la condition réelle de la valve.

Modèle	Cycles	Erreurs	Taux
valve 73 %	40	0	0.000
valve 80 %	42	0	0.000
valve 90 %	50	29	0.580
valve 100 %	73	0	0.000

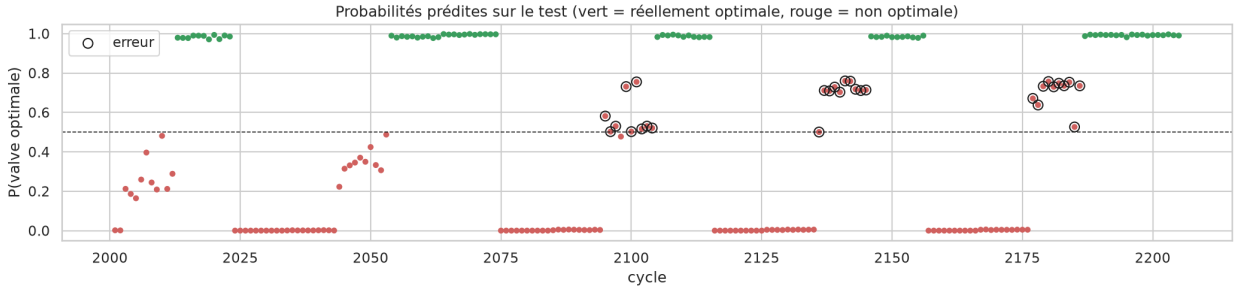


Figure 23 – Probabilité prédite d'une valve optimale pour chaque cycle de test ; les erreurs sont entourées.

Toutes les erreurs concernent des valves à 90 % (léger retard), soit le niveau le plus proche de l'optimum ; les niveaux 80 et 73 % sont parfaitement reconnus. Sur la figure 23, ces cycles ont une probabilité entre 0,5 et 0,75 : ils sont mieux classés que les vrais cycles optimaux (probabilité proche de 1) mais restent au-dessus du seuil. L'explication est le décalage de distribution décrit en section 2 : dans le test, 69 % des cycles ont un accumulateur à 90 bar (33 % à l'entraînement) et tous ont un refroidisseur à 100 %. Ces conditions modifient légèrement le niveau moyen de la pression, donc la valeur de `PS2_w1_mean` sur laquelle repose presque entièrement la régression logistique, et la poussent juste au-dessus du seuil pour les valves à 90 %. Les modèles non linéaires, qui découpent l'espace en plusieurs régions, sont moins sensibles à ce décalage.

Leçon. On ne change pas le modèle après avoir vu le test : ce serait ajuster le choix sur le test, et le score annoncé ne serait plus une estimation honnête. Mais ce résultat montre que, quand le critère de sélection est saturé, il faut un critère plus exigeant (validation à plis contigus en F1, ou validation par groupes de conditions de fonctionnement), et qu'en production un SVM ou une forêt aléatoire serait préférable, avec une surveillance de la calibration du seuil.

7 Interprétation : causes d'une valve non optimale

L'importance par permutation (figure 24) mesure la baisse d'AUC sur le test quand on mélange une variable ; elle ne dépend pas du type de modèle.

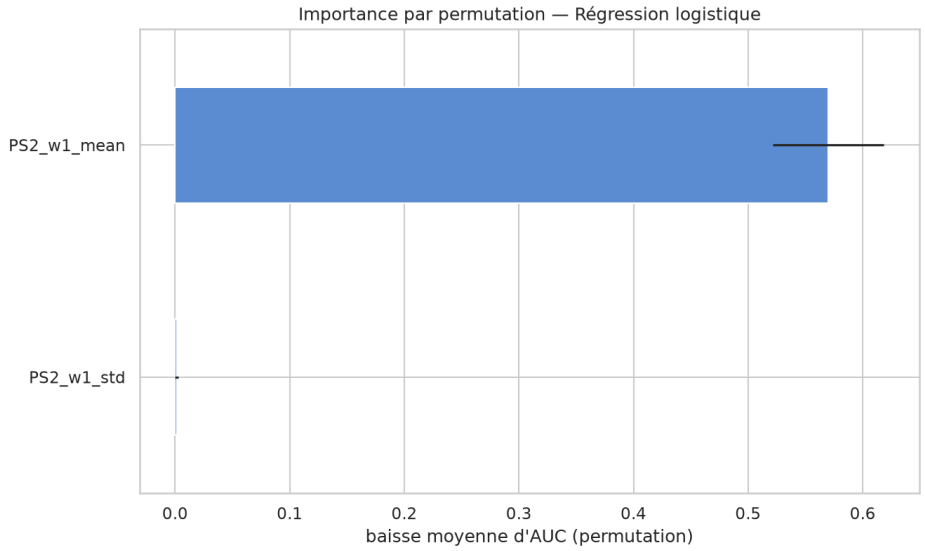


Figure 24 – Importance par permutation des variables du modèle final (test).

Le modèle final repose presque entièrement sur une seule variable, `PS2_w1_mean`, la pression moyenne entre 6 et 12 s : la mélanger fait perdre 0,57 point d'AUC, alors que les autres variables ont une importance quasi nulle (la pénalité L1 a annulé leurs coefficients ou elles sont redondantes avec elle).

Interprétation physique. Cette fenêtre contient la montée en pression de 9,4 s. Si la valve commute en retard, la pression monte plus tard, donc la pression moyenne sur la fenêtre est plus faible. Pour le vérifier, on mesure directement l'instant où PS2 dépasse 50 bar après 5 s (figure 25). Sur l'entraînement, cet instant vaut en moyenne 9.37 s pour une valve à 100 %, 9.44 s à 90 %, 9.54 s à 80 % et 9.67 s à 73 %, avec une dispersion de quelques millisecondes seulement et **sans dépendance à l'état de l'accumulateur**.

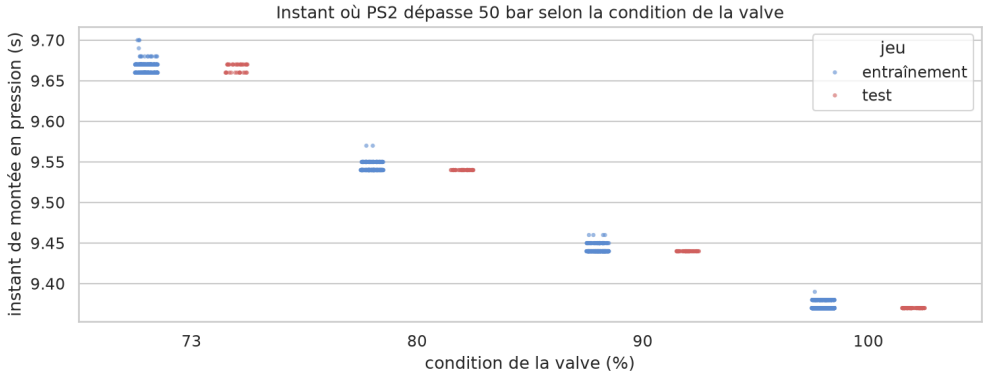


Figure 25 – Instant où la pression PS2 dépasse 50 bar, selon la condition de la valve (entraînement et test).

La cause d'une valve non optimale est donc un retard de commutation : dès le niveau 90 %, la montée en pression arrive environ 70 ms plus tard, et ce retard grandit avec la dégradation. C'est cette signature que tous les modèles exploitent, directement (instant du maximum, raideur de la montée) ou indirectement (moyenne de pression sur la fenêtre, énergie haute fréquence).

Expérience a posteriori. Un classifieur logistique entraîné sur cette seule variable obtient sur le test une accuracy de 1.000 et un F1 de 1.000. Cette expérience a été menée après l'évaluation finale et ne remplace pas le modèle choisi ; elle montre qu'une variable directement liée au mécanisme

physique (un instant, plutôt qu'un niveau de pression) est beaucoup plus robuste aux changements de conditions de fonctionnement.

8 Bonus réalisés

8.1 Tests unitaires

Le dossier `tests/` contient 11 tests `pytest` qui vérifient, sur des signaux simulés : la lecture des fichiers et la numérotation des cycles, la binarisation de la cible, le découpage 2000 / 205 sans recouvrement, la valeur des caractéristiques sur un signal connu, la somme des parts d'énergie spectrale (égale à 1), le filtre de corrélation, le bon fonctionnement des pipelines et l'API de l'application web.

```
python -m pytest -q
```

CAPTURE À INSÉRER

Capture du terminal après `python -m pytest -q` (« 11 passed »).

`fichier attendu : rapport/captures/tests_pytest.png`

Figure 26 – Exécution des tests unitaires.

8.2 Conteneurisation

Le `Dockerfile` construit une image Python 3.11 contenant le code, les dépendances et le modèle entraîné. Elle permet à un tiers d'exécuter le projet sans rien installer :

```
docker build -t valve-ml .
docker run --rm -p 5000:5000 valve-ml                          # application web
docker run --rm valve-ml python -m pytest -q                    # tests
docker run --rm -v "$PWD/data:/app/data" -v "$PWD/rapport:/app/rapport" valve-ml \
  jupyter nbconvert --to notebook --execute notebooks/projet_valve.ipynb --output
    projet_valve.ipynb
```

CAPTURE À INSÉRER

Capture du terminal montrant `docker build` terminé puis `docker run` de l'application.

`fichier attendu : rapport/captures/docker.png`

Figure 27 – Construction et exécution du conteneur Docker.

8.3 Application web

L'application Flask (`app/app.py`) charge le modèle final et renvoie, pour un numéro de cycle donné, la prédiction, la probabilité associée, la condition réelle et si le cycle appartient au jeu d'entraînement ou de test. Une API JSON est aussi disponible : `GET /api/predict?cycle=2050`.

CAPTURE À INSÉRER

Capture du navigateur sur `http://localhost:5000/?cycle=2050` (ou un autre cycle de test) montrant le résultat.

`fichier attendu : rapport/captures/appli_web.png`

Figure 28 – Application web : prédiction pour un cycle du jeu de test.

8.4 Dépôt GitHub

CAPTURE À INSÉRER

Capture de la page d'accueil du dépôt GitHub (arborescence et README). Lien du dépôt : [à compléter].

`fichier attendu : rapport/captures/github.png`

Figure 29 – Dépôt GitHub du projet.

9 Limites et perspectives

— **Décalage de distribution.** Le test correspond à une période où l'accumulateur est fortement dégradé et le refroidisseur en parfait état. Les performances sur le test doivent donc être lues comme une mesure de robustesse à des conditions peu vues à l'entraînement.

— **Critère de sélection saturé.** L'AUC en validation croisée vaut 1 pour presque tous les modèles ; la validation à plis contigus, ou par groupes de conditions de fonctionnement, devrait servir directement à l'optimisation et au choix du modèle.

— **Calibration du seuil.** L'AUC du modèle final sur le test est parfaite : un seuil recalibré périodiquement sur des cycles récents étiquetés corrigerait ses erreurs.
— **Caractéristiques physiques.** Ajouter explicitement l'instant de commutation (et celui du pic de débit) aux variables rendrait les modèles plus robustes.
— **Capteurs.** Seuls PS2 et FS1 sont autorisés ; d'autres capteurs du banc (PS1, EPS1, températures) pourraient améliorer la robustesse.
— **Approches sans extraction manuelle.** Un réseau convolutif 1D sur les signaux bruts, ou des caractéristiques automatiques (*tsfresh*), sont des pistes naturelles.
— **Classification multi-classe.** Prédire le niveau exact (100, 90, 80, 73 %) permettrait de planifier la maintenance avant la panne.

10 Conclusion

Ce projet a mis en place une chaîne complète de maintenance prédictive : contrôle de qualité, construction de la cible binaire, analyse exploratoire, extraction de 90 caractéristiques physiques, sélection de variables sans fuite d'information, comparaison de neuf modèles, évaluation par validation croisée stratifiée, analyse biais/variance et optimisation des hyperparamètres par Grid Search et Random Search. Le modèle retenu, Régression logistique, obtient sur les 205 cycles de test une accuracy de 0.859, un F1 de 0.834 et une AUC de 1.000. Ses erreurs viennent d'un seuil décalé par le changement de conditions de fonctionnement entre les périodes d'entraînement et de test ; les modèles non linéaires optimisés atteignent 100 % sur le test.

Sur le plan métier, l'analyse répond à la question posée : une valve non optimale se reconnaît à un **retard de la montée en pression** dans le cycle, mesurable dès un léger retard de commutation (niveau 90 %) et indépendant de l'état des autres composants. Surveiller cet instant de commutation constitue un indicateur simple et robuste de maintenance prédictive.

A Reproduire les résultats

```
pip install -r requirements.txt
# placer PS2.txt, FS1.txt et profile.txt dans data/
cd notebooks && jupyter nbconvert --to notebook --execute projet_valve.ipynb --output
    projet_valve.ipynb
cd ../rapport && pdflatex rapport.tex && pdflatex rapport.tex
python ../app/app.py
```

Référence

N. Helwig, E. Pignanelli, A. Schütze, *Condition monitoring of a complex hydraulic system using multivariate statistics*, Proc. I2MTC-2015, IEEE International Instrumentation and Measurement Technology Conference, 2015. Jeu de données : UCI Machine Learning Repository, *Condition monitoring of hydraulic systems.*